// Redes de Computadores II · demonstração: TCP e UDP com um miniprotocolo

Atividade individual — Evolução de um miniprotocolo com TCP e UDP

Aluno(a): _____

Turma: _____ **Data:** _____

Você vai modificar o código-base em **dez alterações progressivas**, acrescentando comandos e regras ao protocolo de aplicação. Servidor e cliente rodam **no mesmo computador**, em dois terminais. Você vai **escolher um transporte, TCP ou UDP**, e implementar nele as dez alterações; cliente e servidor precisam continuar compatíveis. A entrega é o **link do repositório no GitHub**.

1 Preparação

1. Crie a pasta `atividade_miniprotocolo`.
2. Salve nela o arquivo fornecido pelo professor.
3. Faça uma cópia e renomeie para `miniprotocolo_nome_sobrenome.py`, com o seu nome.
4. Faça **todas** as alterações somente na cópia.
5. Abra a pasta no VS Code e abra dois terminais.

TERMINAL 1 — SERVIDOR

Inicia primeiro e espera.

TERMINAL 2 — CLIENTE

Conecta e envia as mensagens.

// endereço Mantenha `HOST_SERVIDOR = "127.0.0.1"`. Esse endereço representa o próprio computador: cliente e servidor são processos diferentes na mesma máquina.

2 Como executar servidor e cliente

TERMINAL 1 — SERVIDOR

```
py miniprotocolo_nome_sobrenome.py
```

TCP: escolha `1 - Servidor TCP` e espere:

```
Servidor TCP esperando em
127.0.0.1:9000
```

UDP: escolha `3 - Servidor UDP`.

TERMINAL 2 — CLIENTE

```
py miniprotocolo_nome_sobrenome.py
```

TCP: escolha `2 - Cliente TCP` · UDP: `4 - Cliente UDP`.

Quando pedir o IP, digite `127.0.0.1`. Depois, teste:

ENTRADA (CLIENTE)	RESPOSTA (SERVIDOR)
`ECO|Olá`	`RESPOSTA|Olá`

Para encerrar: `SAIR|`

// atenção Sempre inicie o servidor antes do cliente. Sem o servidor executando, o cliente não encontra um processo aguardando naquela porta. No teste inicial (seção 3), execute TCP e depois UDP; nas alterações, use só o protocolo escolhido.

3 Teste inicial obrigatório

```
ECO|Olá
MAIUSCULA|redes de computadores
TAMANHO|teste
HORA|
SAIR|
```

Envie as cinco mensagens em **TCP** e depois repita em **UDP**. Só comece a modificar o código quando o original funcionar.

☐ Original funciona em TCP ☐ Original funciona em UDP

Escolha do protocolo: ☐ **TCP** — opções 1 e 2 do menu ☐ **UDP** — opções 3 e 4 do menu

Marque um. As dez alterações e os testes serão feitos nele.

4 Regras

- Atividade individual, só com o código-base.
- Sem classes e sem bibliotecas externas.
- Não apague os comandos originais.
- Mantenha os quatro modos do menu; implemente as alterações no protocolo escolhido.
- Não troque o programa por outro código diferente.
- Faça as alterações na ordem; elas são cumulativas.
- Salve e teste depois de cada alteração.
- Só comece uma alteração depois de testar a anterior.
- Não copie a solução de outro estudante.
- Comente cada trecho modificado.

```
# ALTERAÇÃO 01 — comando MINUSCULA
```

5 Dez alterações obrigatórias

01 Alteração 01 — Comando `MINUSCULA` simples

Converter o conteúdo para letras minúsculas.

ENTRADA (CLIENTE)

```
MINUSCULA|REDES DE COMPUTADORES
```

RESPOSTA (SERVIDOR)

```
RESPOSTA|redes de computadores
```

Onde modificar: `processar_mensagem()`

Teste: as entradas acima no protocolo escolhido (seção 6).

☐ Implementado ☐ Testado no protocolo escolhido

02 Alteração 02 — Comando `INVERTER`

simples

Devolver o texto invertido, sem bibliotecas externas.

ENTRADA (CLIENTE)

```
INVERTER|socket
```

RESPOSTA (SERVIDOR)

```
RESPOSTA|tekcos
```

Onde modificar: `processar_mensagem()`

Teste: as entradas acima no protocolo escolhido (seção 6).

☐ Implementado ☐ Testado no protocolo escolhido

03 Alteração 03 — Comando `PALAVRAS`

simples

Contar as palavras do conteúdo. Considere palavras separadas por espaços.

ENTRADA (CLIENTE)

```
PALAVRAS|redes de computadores
```

RESPOSTA (SERVIDOR)

```
RESPOSTA|O texto possui 3 palavras
```

Onde modificar: `processar_mensagem()`

Teste: as entradas acima no protocolo escolhido (seção 6).

☐ Implementado ☐ Testado no protocolo escolhido

04 Alteração 04 — Comando `SOMA`

simples → intermediário

Receber dois números separados por vírgula e devolver a soma. Deve aceitar decimais. Dados inválidos geram erro **sem encerrar o servidor**.

ENTRADA (CLIENTE)

```
SOMA|10,5
```

RESPOSTA (SERVIDOR)

```
RESPOSTA|15
```

ENTRADA (CLIENTE)

```
SOMA|2.5,3.5
```

RESPOSTA (SERVIDOR)

```
RESPOSTA|6.0
```

ENTRADA (CLIENTE)

```
SOMA|dez,cinco
```

RESPOSTA (SERVIDOR)

```
ERRO|Informe dois números separados por vírgula
```

Onde modificar: `processar_mensagem()`

Teste: as entradas acima no protocolo escolhido (seção 6).

☐ Implementado ☐ Testado com inteiros ☐ Testado com decimais ☐ Testado com valores inválidos

05 Alteração 05 — Comando `PING`

simples

Verificar se o servidor está respondendo. O comando não recebe conteúdo.

ENTRADA (CLIENTE)

```
PING|
```

RESPOSTA (SERVIDOR)

```
PONG|Servidor ativo
```

Onde modificar: `processar_mensagem()`

Teste: as entradas acima no protocolo escolhido (seção 6).

- [ ] Implementado
- [ ] Testado no protocolo escolhido

06 Alteração 06 — Comando `AJUDA`

simples → intermediário

Informar todos os comandos disponíveis, em uma linha. Atualize também o menu exibido ao usuário.

ENTRADA (CLIENTE)

```
AJUDA|
```

RESPOSTA (SERVIDOR)

```
RESPOSTA|ECO, MAIUSCULA, MINUSCULA,
INVERTER, TAMANHO, PALAVRAS, SOMA,
HORA, PING, AJUDA, SAIR
```

Onde modificar: `processar_mensagem()` e `mostrar_comandos()`

Teste: as entradas acima no protocolo escolhido (seção 6).

- [ ] Implementado
- [ ] Menu atualizado
- [ ] Testado no protocolo escolhido

07 Alteração 07 — Validar conteúdo obrigatório

intermediário

Exigem conteúdo: `ECO`, `MAIUSCULA`, `MINUSCULA`, `INVERTER`, `TAMANHO`, `PALAVRAS`, `SOMA`. Não recebem conteúdo: `HORA`, `PING`, `AJUDA`, `SAIR`. Uma mensagem inválida não encerra o servidor.

ENTRADA (CLIENTE)

```
MAIUSCULA|
```

RESPOSTA (SERVIDOR)

```
ERRO|O comando MAIUSCULA precisa de
conteúdo
```

ENTRADA (CLIENTE)

```
HORA|texto
```

RESPOSTA (SERVIDOR)

```
ERRO|O comando HORA não recebe
conteúdo
```

Onde modificar: `processar_mensagem()`

Teste: as entradas acima no protocolo escolhido (seção 6).

- [ ] Conteúdo obrigatório validado
- [ ] Conteúdo inesperado validado
- [ ] Servidor continua após o erro

08 Alteração 08 — Normalizar o comando

intermediário

Aceitar o comando em minúsculas, maiúsculas e com espaços antes ou depois do nome. Use `strip()` e `upper()` **somente no comando**, nunca no conteúdo.

ENTRADA (CLIENTE)

```
ECO|Olá
eco|Olá
Eco|Olá
 eco |Olá
```

RESPOSTA (SERVIDOR)

```
RESPOSTA|Olá   (nas quatro entradas)
```

Onde modificar: trecho que separa comando e conteúdo

Teste: as entradas acima no protocolo escolhido (seção 6).

- [ ] Minúsculas aceitas
- [ ] Maiúsculas aceitas
- [ ] Espaços removidos do comando
- [ ] Conteúdo preservado

09 Alteração 09 — Padronizar as respostas

intermediário

Sucesso: `OK|COMANDO|RESULTADO`. Erro: `ERRO|CODIGO|DESCRICAO`. Atualize todos os comandos para o novo padrão.

ENTRADA (CLIENTE)

```
ECO|Olá
PING|
```

RESPOSTA (SERVIDOR)

```
OK|ECO|Olá
OK|PING|Servidor ativo
```

ENTRADA (CLIENTE)

```
TESTE|x
ECO|
HORA|x
SOMA|a,b
sem barra
```

RESPOSTA (SERVIDOR)

```
ERRO|COMANDO_DESCONHECIDO|TESTE
ERRO|CONTEUDO_AUSENTE|O comando ECO
precisa de conteúdo
ERRO|CONTEUDO_INESPERADO|O comando
HORA não recebe conteúdo
ERRO|NUMEROS_INVALIDOS|Informe dois
números válidos
ERRO|FORMATO_INVALIDO|Use
COMANDO|CONTEUDO
```

Onde modificar: `processar_mensagem()` e todas as respostas

Teste: as entradas acima no protocolo escolhido (seção 6).

- [ ] Sucesso padronizado
- [ ] Erros padronizados
- [ ] Todos os comandos atualizados
- [ ] Protocolo escolhido funcionando

10 Alteração 10 — Identificador da solicitação

intermediário

O cliente acrescenta automaticamente um número a cada mensagem. Você continua digitando `ECO|Olá`; pela rede segue `ID|COMANDO|CONTEUDO`. O servidor devolve o mesmo identificador. Para separar os três campos, use `mensagem.split("|", 2)`.

ENTRADA (CLIENTE)

```
1|ECO|Olá   (digitado: ECO|Olá)
```

RESPOSTA (SERVIDOR)

```
1|OK|ECO|Olá
```

ENTRADA (CLIENTE)

```
2|TESTE|x
```

RESPOSTA (SERVIDOR)

```
2|ERRO|COMANDO_DESCONHECIDO|TESTE
```

1. Criar um contador iniciado em `1`.
2. Acrescentar o número antes da mensagem.
3. Enviar, receber a resposta e aumentar o contador.
4. **Não remova** o `\n` delimitador (TCP) nem o envio de um datagrama por mensagem (UDP).

Onde modificar: cliente do protocolo escolhido, interpretação no servidor e criação das respostas

Teste: as entradas acima no protocolo escolhido (seção 6).

- [ ] Contador no cliente escolhido
- [ ] ID recebido pelo servidor
- [ ] ID devolvido na resposta
- [ ] Testado com várias mensagens

6 Procedimento após cada alteração

1. Salve o arquivo.
2. Terminal 1: inicie o servidor do protocolo escolhido.
3. Terminal 2: inicie o cliente e informe `127.0.0.1`.
4. Teste o novo comportamento.
5. Envie `SAIR|` e reinicie os programas.
6. Corrija os erros e teste de novo.
7. Marque a alteração como concluída.
8. Faça um commit (seção 9).

> // atenção Não basta o código executar. Confira se a resposta recebida corresponde exatamente à regra definida pelo protocolo.

7 Testes finais

Com as dez alterações prontas, envie as mensagens abaixo. Use o protocolo escolhido; o cliente acrescenta os identificadores sozinho; termine com `SAIR|`.

Entrada	Resultado esperado	Feito
`ECO|Olá`	Eco com resposta padronizada	☐
`MINUSCULA|REDES DE COMPUTADORES`	Texto em minúsculas	☐
`INVERTER|socket`	Texto invertido	☐
`PALAVRAS|redes de computadores`	Três palavras	☐
`SOMA|10,5`	Resultado 15	☐
`PING|`	Servidor ativo	☐
`AJUDA|`	Lista de comandos	☐
`MAIUSCULA|`	Erro de conteúdo ausente	☐
`HORA|texto indevido`	Erro de conteúdo inesperado	☐
`COMANDO_INEXISTENTE|teste`	Erro de comando desconhecido	☐

8 Registro das alterações

```
"""
ALUNO:
TURMA:
DATA:
PROTOCOLO ESCOLHIDO: TCP ou UDP

REGISTRO DAS ALTERAÇÕES

ALTERAÇÃO 01
O que foi implementado:
Entrada testada:
Saída obtida:

ALTERAÇÃO 02
...
Continue até a ALTERAÇÃO 10.
"""
```

Cole este bloco **no início do arquivo Python** e preencha uma entrada por alteração.

Além do registro, comente os trechos alterados:

```
# ALTERAÇÃO 03 — contagem de palavras
```

Não é necessário relatório separado nem capturas de tela.

9 Entrega

A entrega é **somente o link do repositório no GitHub**. O repositório deve conter o arquivo `miniprotocolo_nome_sobrenome.py` com o registro e as dez alterações.

1. No GitHub, crie um repositório **público e vazio** (sem README, licença ou `.gitignore`) chamado `miniprotocolo-nome-sobrenome`.
2. No terminal, dentro da pasta `atividade_miniprotocolo`, execute:

```
git init
git remote add origin https://github.com/usuario/miniprotocolo-nome-sobrenome.git
git add miniprotocolo_nome_sobrenome.py
git commit -m "Implementa as alterações do miniprotocolo"
git branch -M main
git push -u origin main
```

- Troque usuario e o nome do repositório pela URL copiada do GitHub.
- Confira no navegador se o arquivo aparece no repositório.
- A cada alteração concluída: `git add`, `git commit` e `git push`.
- Envie apenas o link, por exemplo: `https://github.com/usuario/miniprotocolo-nome-sobrenome`.

Link do repositório: _____

10 Checklist final

- [ ] Realizei a atividade individualmente.
- [ ] O programa inicia sem erro.
- [ ] Testei o cliente no Terminal 2.
- [ ] As quatro opções do menu funcionam.
- [ ] Implementei as dez alterações.
- [ ] As respostas seguem o formato definido.
- [ ] Implementei e testei no protocolo escolhido (TCP ou UDP).
- [ ] O arquivo possui comentários.
- [ ] O repositório no GitHub é público.
- [ ] Entreguei somente o link do repositório.
- [ ] O arquivo possui meu nome.
- [ ] Testei o servidor no Terminal 1.
- [ ] Usei o endereço `127.0.0.1` .
- [ ] Os comandos originais continuam funcionando.
- [ ] Mensagens inválidas não encerram o servidor.
- [ ] O identificador aparece nas mensagens.
- [ ] As alterações estão documentadas.
- [ ] O nome do arquivo segue o padrão.
- [ ] Fiz `git push` e o arquivo aparece no GitHub.